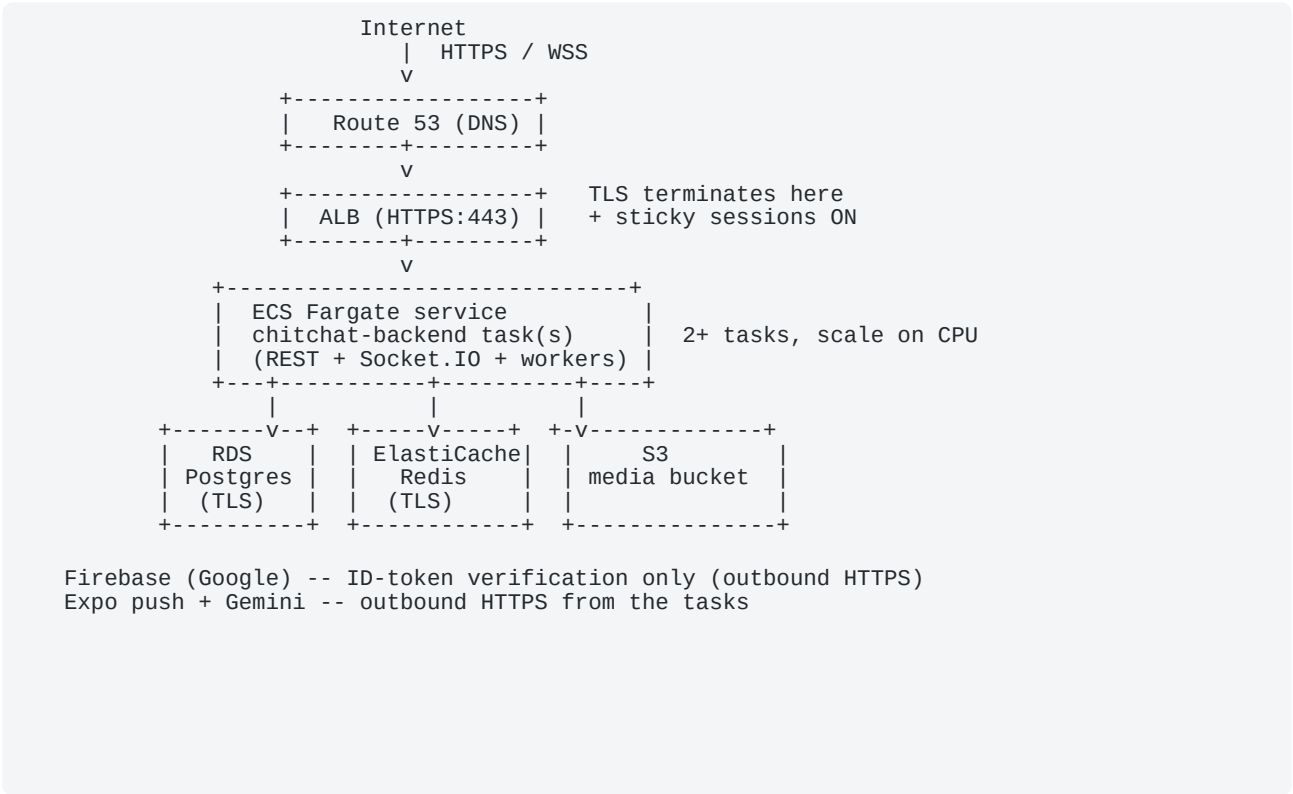
Deploying the ChitChatBusiness backend to AWS

This guide deploys the `server/` backend (Fastify REST API + Socket.IO gateway) to AWS. It is written against the code as it exists today: a single Node 20 process that needs **PostgreSQL**, **Redis**, an **S3-compatible object store**, and a **Firebase service account** for ID-token verification.

> The app code is production-quality. The pieces you provide are infrastructure (database, cache, storage, TLS, secrets) and the deploy wiring below.

1. Target architecture



Why these services

Need	AWS service	Notes
Run the container	ECS Fargate	No servers to manage. App Runner is a simpler alternative but ALB sticky sessions give you more control for Socket.IO.

Load balancing + TLS	ALB (Application Load Balancer)	Terminates HTTPS/WSS, supports WebSocket upgrade, and sticky sessions (required — see §6).
Database	RDS PostgreSQL	TLS required; the app now supports it via DATABASE_SSL.
Cache / pub-sub / jobs	ElastiCache for Redis	Mandatory — presence, the Socket.IO adapter, rate limits, and BullMQ all depend on it. Must be managed Redis, not in-process.
Media storage	S3	Presigned PUT/GET. Flip S3_FORCE_PATH_STYLE=false and drop the custom endpoint.
Secrets	Secrets Manager	Firebase service account, DB/Redis URLs, S3 keys, Gemini key.
Image registry	ECR	Holds the Docker image built from server/Dockerfile.
DNS	Route 53	api.yourdomain.com !’ ALB.
TLS cert	ACM	Free public cert for the ALB.

2. Prerequisites

- AWS account + AWS CLI configured (`aws configure`)
- Docker installed (image already builds — see `server/Dockerfile`)
- A domain you control (for `api.yourdomain.com`)
- The Firebase service-account JSON (currently `server/firebase-service-account.json`)

Pick a region (examples below use `us-east-1`) and an account id placeholder `<ACCOUNT_ID>`.

3. Build and push the image to ECR

```
cd server

# 3.1 Create the repository (once).
aws ecr create-repository --repository-name chitchat-backend --region us-east-1

# 3.2 Log in.
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin \
    <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com

# 3.3 Build for the Fargate architecture (x86_64 is the default platform).
docker build --platform linux/amd64 -t chitchat-backend:latest .

# 3.4 Tag and push.
docker tag chitchat-backend:latest \
  <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/chitchat-backend:latest
docker push <ACCOUNT_ID>.dkr.ecr.us-east-1.amazonaws.com/chitchat-backend:latest
```

> If you run Fargate on ARM (Graviton) use `--platform linux/arm64` and set the task's `runtimePlatform` CPU architecture to `ARM64`.

---

4. Provision the data stores

4.1 Networking

- Use a VPC with **2 public subnets** (ALB) and **2 private subnets** (Fargate tasks, RDS, ElastiCache) across two AZs.
- NAT Gateway (or VPC endpoints) so private tasks can reach Firebase/Expo/ECR.
- Security groups:
  - `sg-alb` : inbound 443 from the internet.
  - `sg-app` : inbound 4000 **from** `sg-alb` **only**.
  - `sg-rds` : inbound 5432 **from** `sg-app` **only**.
  - `sg-redis` : inbound 6379 **from** `sg-app` **only**.

4.2 RDS PostgreSQL

- Engine: PostgreSQL 16 (matches the dev `postgres:16-alpine`).
- Place in private subnets, attach `sg-rds`.
- **Enforce TLS**: parameter group `rds.force_ssl = 1`.
- Note the endpoint !' `DATABASE_URL=postgres://USER:PASS@<endpoint>:5432/chitchat`.

4.3 ElastiCache for Redis

- Redis 7 (matches dev `redis:7-alpine`).
- Enable **in-transit encryption (TLS)** !' connect with `rediss://`.
- Place in private subnets, attach `sg-redis`.
- Note the primary endpoint !' `REDIS_URL=rediss://<endpoint>:6379`.

4.4 S3 bucket

```
aws s3api create-bucket --bucket chitchat-media-prod --region us-east-1
# Block public access (presigned URLs grant time-limited access instead).
aws s3api put-public-access-block --bucket chitchat-media-prod \
  --public-access-block-configuration \

BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

If the mobile app uploads/downloads directly via presigned URLs from browsers, add a CORS config to the bucket allowing `PUT` / `GET` from your app origins.

> **Prefer an IAM task role over static keys.** The S3 client in `src/media/presign.ts` reads `S3_ACCESS_KEY_ID` / `S3_SECRET_ACCESS_KEY`. For a first deploy, create an IAM user scoped to the bucket and put its keys in Secrets Manager. To use the task role instead (no static keys), that file needs a small change to drop the explicit `credentials` and let the SDK use the ambient role — say the word and I'll make it.

5. Store secrets in Secrets Manager

Create one JSON secret, e.g.
`chitchat/backend`
, with:

```
{
  "DATABASE_URL": "postgres://USER:PASS@<rds-endpoint>:5432/chitchat",
  "REDIS_URL": "rediss://<elasticache-endpoint>:6379",
  "S3_ACCESS_KEY_ID": "AKIA...",
  "S3_SECRET_ACCESS_KEY": "...",
  "FIREBASE_SERVICE_ACCOUNT_JSON": "{\"type\":\"service_account\", ... }",
  "GEMINI_API_KEY": "..."
}
```

The task definition maps each key to a container env var via
`secrets`
(so they
never appear in plaintext in the task def). Non-secret config stays in
`environment`
(see §7).

> `FIREBASE_SERVICE_ACCOUNT_JSON`
> is the raw JSON string of your service account.
> This is why the server supports the
> `_JSON`
> variant — no file is baked into the
> image (
> `.dockerignore`
> excludes it).

6. ALB and sticky sessions (important for Socket.IO)

Socket.IO starts a connection over HTTP long-polling and upgrades to WebSocket.
Across multiple tasks, the polling handshake
must land on the same task
or
the upgrade fails intermittently. Two ways to handle it:

1. **Enable target-group stickiness**
   (recommended): on the ALB target group set
   stickiness type
   `app_cookie`
   or
   `lb_cookie`
   , duration ~1 hour.
2. **Or force WebSocket-only**
   on the client (
   `transports: ['websocket']`
   ),
   which skips the polling phase. Check the app's Socket.IO client config before
   relying on this.

Even with the Redis adapter (which handles cross-task message fan-out), the
handshake
affinity above is still required.

ALB setup:

- Listener on
  443
  with an

ACM certificate for `api.yourdomain.com`.

- (Optional) Listener on 80 that redirects to 443.
- Target group: protocol HTTP, port **4000**, target type `ip` (Fargate).
- **Health check path:** `/health` (the app returns 200 only when Postgres and Redis are both reachable).
- Idle timeout: raise to **300s+** so idle WebSocket connections aren't dropped.

---

7. ECS task definition (Fargate)

Key points for the container:

- Image: the ECR URI from §3.
- Port mapping: container port **4000**.
- `environment` (non-secret):
  - `NODE_ENV=production`
  - `PORT=4000`
  - `DATABASE_SSL=require` (use `verify` + `DATABASE_CA_CERT` for full verification)
  - `S3_REGION=us-east-1`
  - `S3_ENDPOINT=https://s3.us-east-1.amazonaws.com`
  - `S3_BUCKET=chitchat-media-prod`
  - `S3_FORCE_PATH_STYLE=false`
  - `CORS_ORIGINS=https://app.yourdomain.com` (your real app origins — never `*`)
  - `FIREBASE_PROJECT_ID=chitchatbusiness-880ce`
  - `GEMINI_MODEL=gemini-3.8-flash` (only if you use AI minutes)
- `secrets` (from Secrets Manager §5): `DATABASE_URL`, `REDIS_URL`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, `FIREBASE_SERVICE_ACCOUNT_JSON`, and `GEMINI_API_KEY` (optional).

- Logging:
  `awslogs`
  driver !' CloudWatch Logs. The app already emits structured JSON (pino), so CloudWatch Logs Insights can query it directly.
- CPU/memory: start at
  0.5 vCPU / 1 GB
  per task; adjust from metrics.

DATABASE_SSL
quick reference

Value	Behaviour	When
disable	No TLS	Local dev only
require	TLS on, cert chain not verified	Easiest RDS setup
verify	TLS on, chain verified against system CAs + DATABASE_CA_CERT	Strongest; set DATABASE_CA_CERT to the RDS global CA bundle path/PEM

Redis TLS is automatic when
`REDIS_URL`
uses
`rediss://`
(or set
`REDIS_TLS=true`
).

---

8. Database migrations

Migrations are
not
applied automatically on boot — run them explicitly. The image has a dedicated entry point:

```
node dist/db/migrate.js up
```

Run it as a
one-off ECS task
(same task definition, overridden command) before each deploy that ships schema changes:

```
aws ecs run-task \
  --cluster chitchat \
  --task-definition chitchat-backend \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[subnet-priv-a,subnet-priv-
b],securityGroups=[sg-app],assignPublicIp=DISABLED}" \
  --overrides '{"containerOverrides":[{"name":"chitchat-backend","command":["node","dist/db/
migrate.js","up"]}]}'
```

`node-pg-migrate`
records applied migrations in
`pgmigrations`
, so re-running is safe and idempotent. (It is now a runtime dependency so it's present in the image.)

---

9. ECS service

- Cluster: Fargate.
- Service with **desired count "e 2** (HA across AZs, and lets the Redis adapter earn its keep).
- Attach to the ALB target group from §6.
- Private subnets, `sg-app` .
- Auto scaling: target tracking on **CPU ~60%** (and/or ALB request count). Min 2, max to taste.
- Deployment: rolling with `minimumHealthyPercent=100` , `maximumPercent=200` for zero-downtime. The app's graceful shutdown (SIGTERM !' drain sockets/jobs/pool within 15s) cooperates with ECS task draining; set the ALB deregistration delay to ~20s.

---

10. Point the app at the backend

The mobile/web app reads (from the repo-root `.env`):

- `EXPO_PUBLIC_API_URL=https://api.yourdomain.com`
- `EXPO_PUBLIC_SOCKET_URL=https://api.yourdomain.com` (defaults to the API URL)

Make sure every app origin is listed in the backend `CORS_ORIGINS` .

---

11. Post-deploy verification

```
# Health (expects {"status":"ok", ...}).
curl -s https://api.yourdomain.com/health | jq

# TLS (ALB cert).
curl -sI https://api.yourdomain.com/health | head -1
```

- Confirm a real sign-in produces a valid Firebase token the server accepts.
- Open two app clients and confirm realtime messaging works **across tasks**

  (scale to "e2 tasks first, then message between them).
- Watch CloudWatch Logs for `ChitChat backend listening` and no repeated connection errors to RDS/ElastiCache.
- Trigger a media upload to confirm presigned S3 URLs work with `S3_FORCE_PATH_STYLE=false` .

---

12. Operational notes

- **Scaling Socket.IO**
  : handled by the Redis adapter for fan-out + ALB
  stickiness for handshakes. No extra work to add tasks.
- **Workers run in-process**
  : push/cleanup/call-timeout BullMQ workers live in
  the same task as the API. Fine to start. If push/cleanup volume grows, split
  them into a separate ECS service (same image, a worker-only entry point) so
  they don't compete with request handling — ask and I'll add that entry point.
- **Secrets rotation**
  : rotate the Firebase service account / DB creds in
  Secrets Manager; new tasks pick them up on next deploy.
- **Backups**
  : enable automated RDS snapshots; enable S3 versioning on the media
  bucket if you want delete protection.
- **Cost levers**
  : Fargate task size, RDS instance class, single-node vs.
  replicated ElastiCache, NAT Gateway. Start small; scale on metrics.

---

13. What changed in the codebase to support this

- `src/config/env.ts` — added `DATABASE_SSL`, `DATABASE_CA_CERT`, `REDIS_TLS`.
- `src/db/pool.ts` — Postgres pool now applies TLS from `DATABASE_SSL` (`disable` / `require` / `verify`); exported `sslConfig()`.
- `src/db/migrate.ts` — migrations honour the same TLS settings.
- `src/redis/client.ts` — Redis clients + a shared `bullConnection()` helper enable TLS for `rediss://` / `REDIS_TLS=true`.
- `src/jobs/queues.ts`, `src/jobs/scheduler.ts` — use `bullConnection()`.
- `package.json`

— `node-pg-migrate` moved to runtime dependencies (migrations run inside the container).
- `Dockerfile`, `.dockerignore` — production multi-stage image; secrets excluded; `HEALTHCHECK` on `/health`; `dumb-init` for signal handling.
- `.env.example` — documents the new TLS variables.